ТЕКСТОВЫЙ РЕДАКТОР С ПОДСВЕТКОЙ СИНТАКСИСА

(Консольный или GUI-редактор с поддержкой языков программирования)

Егоров Иван Александрович

ЭФБО-02-24

ВВЕДЕНИЕ

1.1 Цель

Создать текстовый редактор с поддержкой подсветки синтаксиса для нескольких языков программирования, предназначенный для быстрого и удобного написания и редактирования кода.

1.2 Область действия (Scope)

В рамках проекта (In Scope). В разработку входят:

- Создание, открытие, чтение, редактирование и сохранение файлов различных форматов (.txt, .py, .cpp, .cs)
- Подсветка синтаксиса для нескольких языков программирования (Python, C++, C#)
- Отображение номеров строк
- Базовый пользовательский интерфейс

Вне рамок проекта (Out of scope). В разработку не входит:

- Полноценный функционал IDE (компиляция, запуск кода, отладка, отображение ошибок синтаксиса)
- Поддержка различных расширений
- Поддержка системы контроля версий (Git)

1.3 Определения и аббревиатуры

Термин	Описание
Подсветка синтаксиса	Выделение элементов кода (слов, строк) разными цветами для удобства написания, редактирования и чтения
IDE	Integrated Development Environment — среда разработки, включающая редактор кода, отладчик, компилятор и прочее инструменты для разработки
ПО	Программное обеспеченье
UML	Unified Modeling Language — унифицированный язык визуального моделирования

ОБЩЕЕ ОПИСАНИЕ

2.1 Описание продукта

Разрабатываемый продукт представляет собой текстовый редактор для работы с исходным кодом. Редактор обладает достаточным функционалом для написания, чтения и редактирования файлов программ. Редактор оснащен функцией подсветки синтаксиса для удобного визуального восприятия кода.

2.2 Функции продукта

Система обладает следующими функциями:

- Работа с файлами (создание, открытие, редактирование, сохранение)
- Подсветка синтаксиса
- Отображение нумерации строк

2.3 Характеристики пользователей

Основная целевая аудитория:

- Начинающие разработчики ПО
- Студенты IT-направлений
- Системные администраторы

Уровень подготовки:

- Базовые навыки работы с компьютером
- Базовые знания написания кода (не обязательно)

Основные потребности:

- Простота освоения
- Быстрая работа с кодом
- Подсветка синтаксиса
- Поддержка различных форматов файлов

ДЕТАЛЬНЫЕ (СПЕЦИФИЧЕСКИЕ) ТРЕБОВАНИЯ

3.1 Методология сбора требований

Для проекта использовался комбинированный подход к сбору требований.

Применённые методы:

- **Анализ аналогов**

  Были изучены существующие решения на рынке текстовых редакторов и IDE, что позволило понять ожидания пользователей.

- **User Stories**

  Требования формулировались исходя от реальных потребностей пользователей.

- **Декомпозиция требований**

  Все требования были структурированы по уровням: бизнес, пользовательские, функциональные и нефункциональные.

3.2 Бизнес-требования

Бизнес-требования, определяющие цели и задачи проекта со стороны заказчика:

- Занять нишу между тяжёлыми IDE и простыми текстовыми редакторами за счёт уникального сочетания низкого потребления ресурсов и подсветкой синтаксиса
- Снизить порог входа для новых пользователей до минимума за счёт интуитивно понятного интерфейса, не требующего обучения
- Обеспечить монетизацию через freemium-модель: базовая версия бесплатна, а платная подписка открывает поддержку дополнительных языков программирования
- Обеспечить работу на маломощном оборудовании, что позволит занять нишу, недоступную конкурентам

3.3 Пользовательские требования

Пользовательские требования описывают задачи, возможность решения которых должен иметь пользователь программы.

Требования в формате User Stories:

- Как пользователь, я хочу открывать файлы для работы с их содержимым

- Как пользователь, я хочу сохранять файлы, чтобы не потерять изменения
- Как пользователь, я хочу видеть подсветку синтаксиса, чтобы легче понимать и ориентироваться в коде
- Как пользователь, я хочу видеть номера строк для удобной ориентации в файле

3.4 Функциональные требования

Функциональные требования описывают выполняющиеся системой действия:

- Система должна обеспечивать открытие файлов
- Система должна обеспечивать редактирование файлов
- Система должна обеспечивать сохранение файлов
- Система должна обеспечивать подсветку синтаксиса в файлах с определенным расширением (в файлах с кодом)
- Система должна определять язык программирования по расширению файла для обеспечения корректной подсветки синтаксиса для конкретного языка программирования
- Система должна обеспечивать отображение нумерации строк

3.5 Нефункциональные требования

Нефункциональные требования описывают качество системы:

- **Производительность**

  Время запуска, закрытия программы, а также открытие файлов и их сохранение, не должно превышать нескольких секунд на устройстве, поддерживающем системные требования программы.

- **Удобство использования**

  Интерфейс должен быть интуитивно понятным и не требовать длительного обучения.

- **Надёжность**

  Система должна корректно сохранять данные, чтобы избежать их потерь.

- **Совместимость**

  Приложение должно корректно работать на поддерживаемых операционных системах.

3.6 Технические требования

- Проект должен быть реализован на языке программирования C++.
- Поддержка операционной системы Windows 10/11.

ДИАГРАММЫ UML

4.1 Диаграмма вариантов использования (Use Case Diagram)

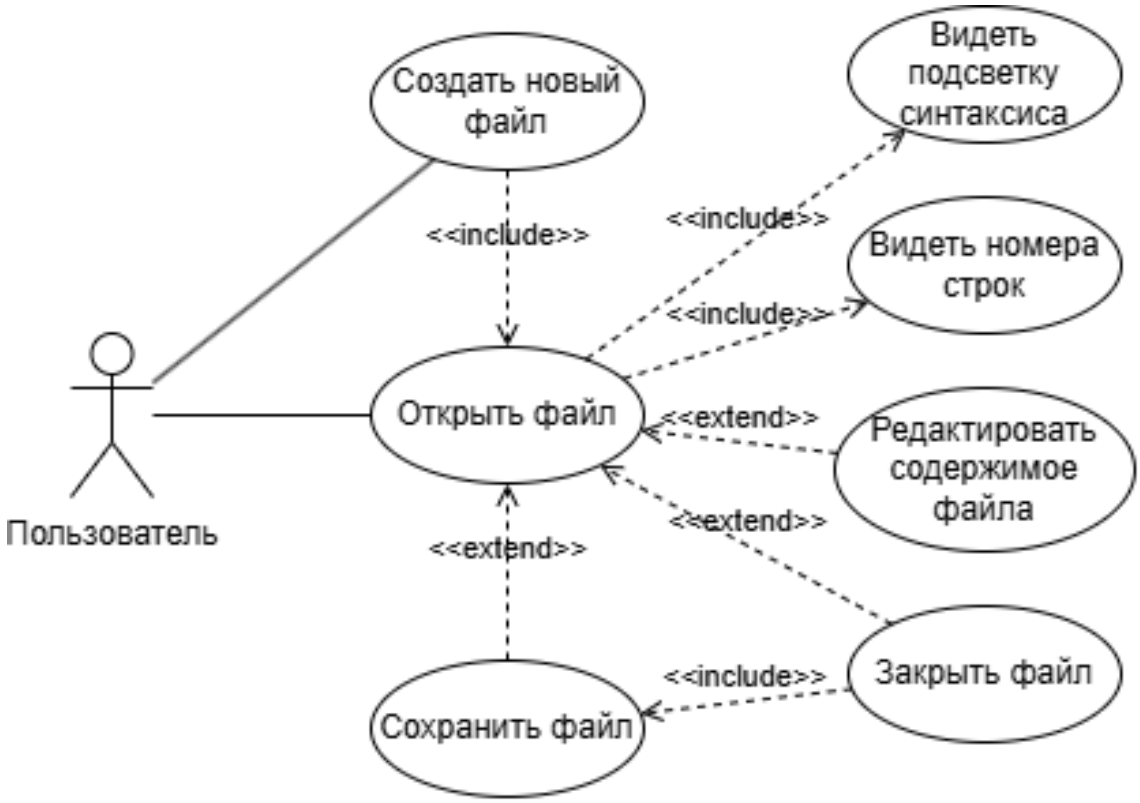

Рисунок 4.1 — Use Case Diagram

4.2 Диаграмма классов (Class Diagram)

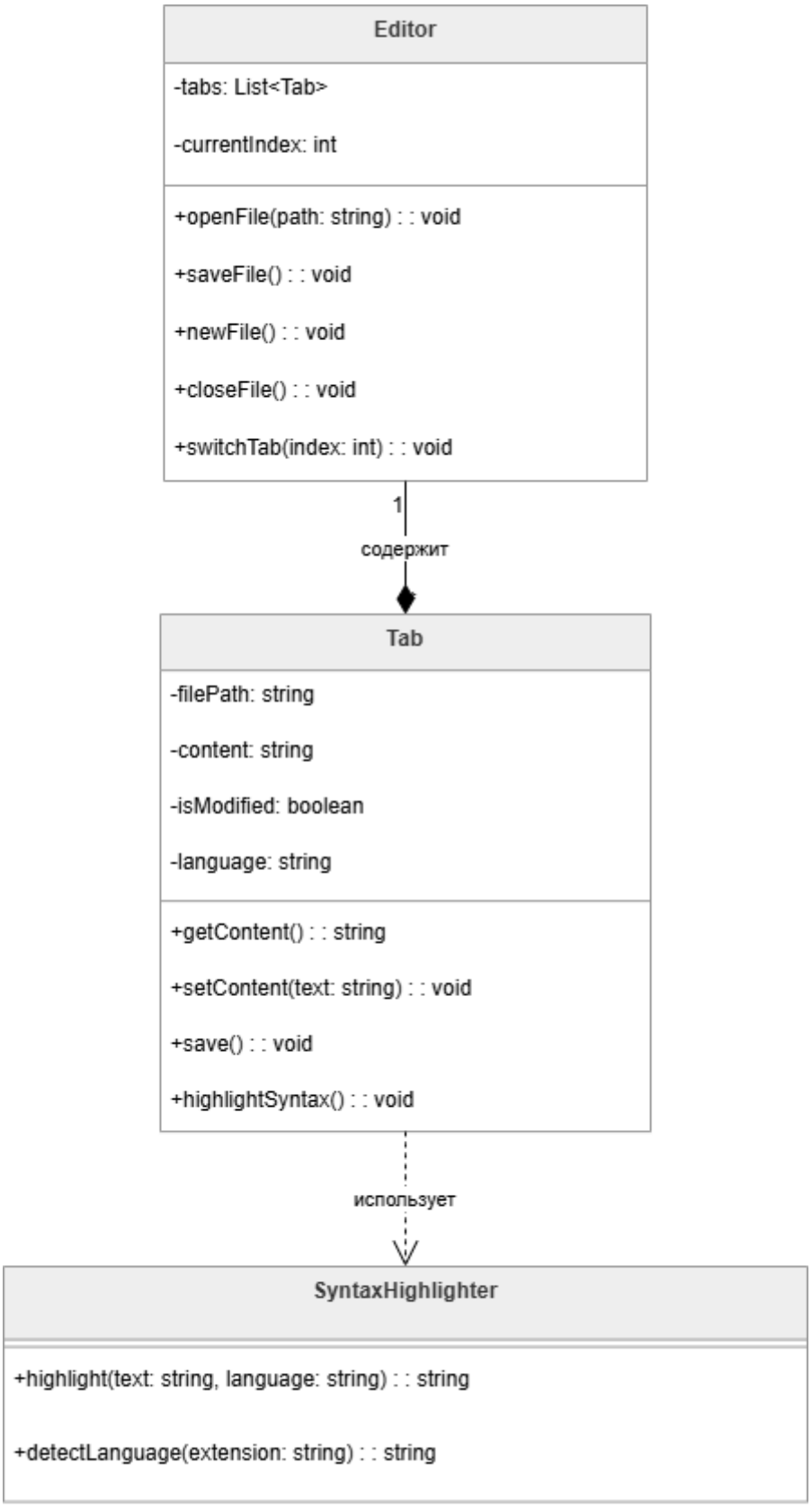

Рисунок 4.2 — Class Diagram

4.3 Диаграмма последовательности (Sequence Diagram)

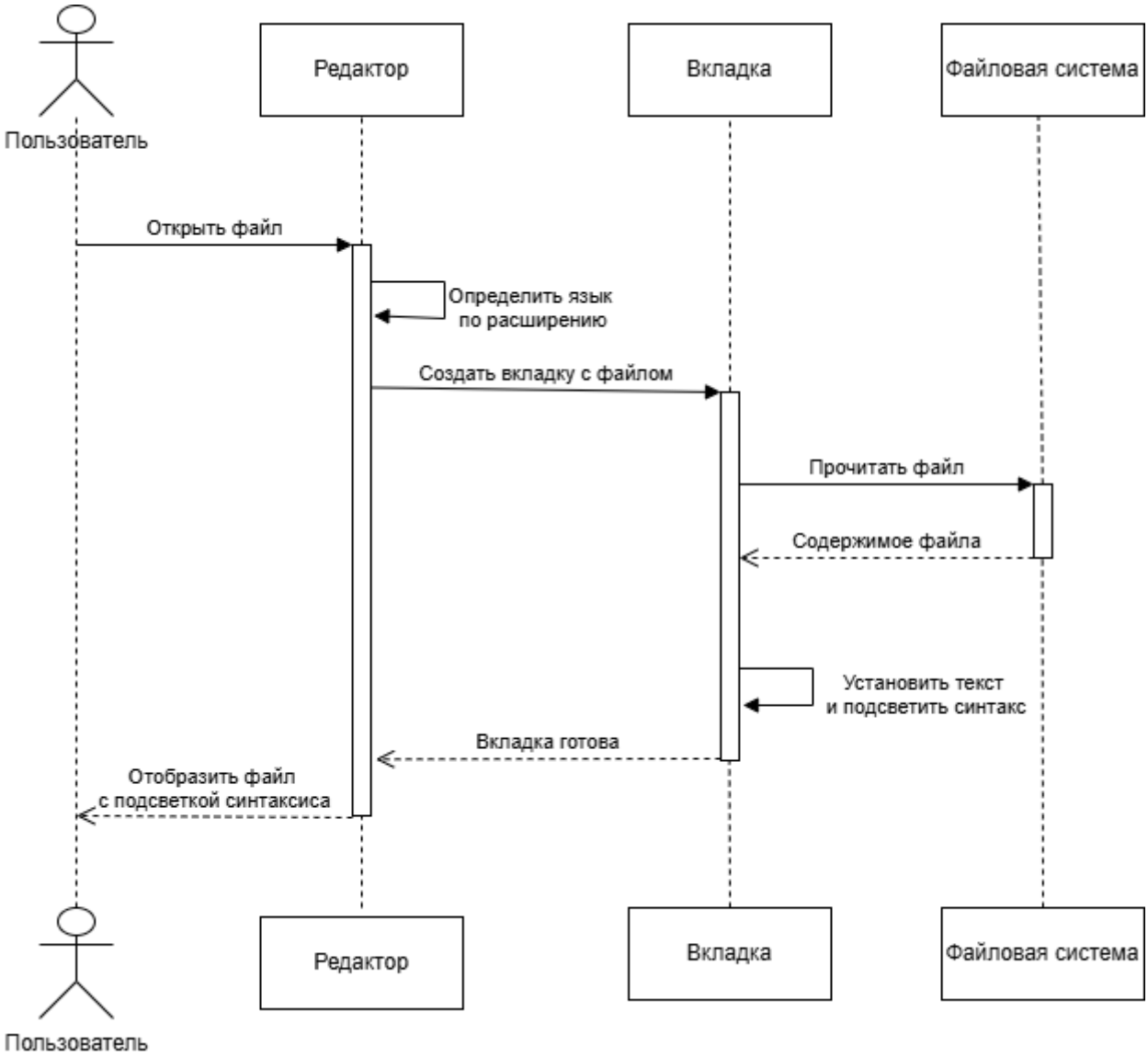

Рисунок 4.3 — Sequence Diagram

Паттерны проектирования

5.1 Выбор паттернов

В ходе анализа архитектуры текстового редактора были рассмотрены различные паттерны проектирования, по итогам были выбраны следующие паттерны:

- **Strategy**

  В базовый функционал редактора входит подсветка синтаксиса для трёх языков программирования, каждый из которых требует собственного алгоритма подсветки. Паттерн позволяет динамически подменять эти алгоритмы без изменения основного кода.
- **Command**

  Для удобства использования текстового редактора можно добавить в него поддержку отмены последнего действия, например, в виде горячих клавиш (Ctrl+Z). Паттерн позволит превратить каждое действие в объект, который можно будет использовать для отмены этого действия.

5.2 Детальное описание применения паттернов

5.2.1 Паттерн Strategy

Данный паттерн применим в классе *SyntaxHighlighter* (Рисунок 4.2), который отвечает за подсветку синтаксиса.

Вместо написания одного метода с проверкой языка создаётся отдельная стратегия под каждый язык, что позволяет легко расширять функционал, путём добавления новых языков без изменения существующего кода.

5.2.2 Паттерн Command

Данный паттерн применим в классе *Editor* (Рисунок 4.2) для реализации отмены действий.

Действия, такие как вставка и удаление символа, превращаются в объекты-команды, которые складываются в стек. При нажатии горячих клавиш (Ctrl+Z) из стека достаётся последняя команда и вызывается её метод, который отменяет действие.